ASSIGNMENT 2

Teacher: Sir Bilal Khan

COURSE: Object Oriented Programming

SECTION: SE-2A, CS-2A, CS-2B, CS-2C

Instructions:

- All assignments should be done individually.
- **Plagiarized work will be given zero marks.**
- **Make sure your code compiles and runs.**
- **Submit your .cpp files. Name your files as** (P25-1234_Name_Section.cpp)
- **Submit PDF Files with outputs as well.**
- **Late submissions will not be entertained.**
- **If there is any confusion make assumptions or ask the TA.**

Rules

- No vectors, lists, or maps
- No fixed-size arrays
- Must resize arrays manually

Must make sure proper memory deallocation

HOSPITAL MANAGEMENT SYSTEM

Design a **menu-driven C++ program** using:

- Classes and Objects
- Constructors
- Static Keyword
- Const Keyword

SYSTEM HIERARCHY

```
Hospital
└── Departments[]
    └── Doctors[]
        └── Patients[]
```

CLASS DESIGN

Patient Class

```
class Patient {
   char  name[40];
   int   age;
   float bill;
};
```

Doctor Class

```
class Doctor {
   char     name[40];
   int      experience;
```

```
    Patient* patients;
    int      patientCount;
    static int TotalPatients;
};
```

Department Class

```
class Department {
    char    DeptName[40];
    Doctor* doctors;
    int     DoctCount;
};
```

Hospital Class

```
class Hospital {
    char       HospitalName[40];
    Department* depts;
    int        DepartmentCount;
};
```

REQUIREMENTS

- Default constructors
- Parameterized constructors
- Static variable for total patients
- Const functions
- Dynamic memory allocation

INPUT POLICY

1. Enter number of departments
2. For each department:
    - Department name
    - Number of doctors
3. For each doctor:
    - Name, Experience
    - Number of patients
4. For each patient:

- Name, Age, Bill

MENU FUNCTIONS

1. Add Department

- Resize array
- Allocate doctors

2. Add Doctor

- Select department
- Resize array

3. Add Patient

- Select department → doctor
- Increment static count

4. Display Data ***(const)***

- Show full hierarchy

5. Department Revenue

- Sum patient bills

6. Most Experienced Doctor

- Find max experience

7. Highest Bill Patient

- Find max bill

8. Increase Bills

- Apply percentage

9. Discharge Patient

- Remove + resize
- Decrement static

10. Remove Empty Doctor

- Deallocate + resize

11. Remove Empty Department

- Deallocate + resize

12. Total Patients

- Show static count

13. Exit Program

- Deallocate ALL memory

SAMPLE OUTPUT FOR MENU

```
===== HOSPITAL MANAGEMENT SYSTEM =====

Enter your choice:
1. Add Department
2. Add Doctor
3. Add Patient
4. Display Hospital Data
5. Calculate Department Revenue
6. Find Most Experienced Doctor
7. Find Patient with Highest Bill
8. Increase Bills by Percentage
9. Discharge Patient
10. Remove Empty Doctor
11. Remove Empty Department
12. Show Total Patients
13. Exit Program

 ====================================

Enter Choice:
```

SMART CITY PARKING MANAGEMENT SYSTEM

The **City of Peshawar** has launched a **Smart Parking Initiative** across multiple zones. Each zone contains several parking lots, and each lot holds a dynamic list of parked vehicles.

The city administration requires a **robust, memory-safe C++ system** to manage this in real time.

The system must:

- Track total vehicles using a **static counter**
- Provide **read-only access using const functions**
- Support **dynamic resizing** for real-time updates

You are the engineer responsible for implementing this system.

OOP CONCEPTS REQUIRED

- Classes and Objects
- Default Constructors
- Parameterized Constructors
- **static keyword** (shared class-level counter)
- **const keyword** (read-only functions)

SYSTEM HIERARCHY

```
City
 └── Zone[] (dynamic array)
     └── ParkingLot[] (dynamic array)
         └── Vehicle[] (dynamic array)
```

CLASS DESIGN

Vehicle Class

```
class Vehicle {
   int    vehicleId;
   char   licensePlate[20];
   char   ownerName[40];
   float  parkingFee;
   int    hoursParked;
   static int totalVehiclesInCity;
};
```

ParkingLot Class

```
class ParkingLot {
   int     lotId;
   char    lotName[40];
   int     capacity;
```

```
    int      currentCount;
    Vehicle* vehicles;
};
```

Zone Class

```
class Zone {
    int        zoneId;
    char       zoneName[40];
    ParkingLot* lots;
    int        lotCount;
};
```

City Class

```
class City {
    char  cityName[40];
    Zone* zones;
    int   zoneCount;
};
```

REQUIREMENTS

- Use **default constructors** to initialize values
- Use **parameterized constructors** for data assignment
- Maintain static int totalVehiclesInCity
- Use **const functions** for display/search
- Use **dynamic memory allocation** for all arrays
- Ensure **proper memory deallocation** (no leaks)

INPUT POLICY

At program start:

1. Enter **city name**
2. Enter **number of zones**
    - Zone ID, Zone Name
    - Number of parking lots
        - Lot ID, Name, Capacity
3. Enter **vehicles**

- Vehicle ID, License Plate, Owner Name
- Hours Parked, Parking Fee

MENU-DRIVEN FUNCTIONS

1. Add Zone

- Resize zone array (+1)
- Add using constructor

2. Add Parking Lot

- Select zone
- Resize lot array
- Allocate vehicles dynamically

3. Park Vehicle

- Select zone → lot
- Resize if full
- Increment static counter

4. Display City Report ***(const)***

- Show all zones, lots, vehicles

5. Exit Vehicle

- Remove vehicle
- Shift array
- Decrement static counter

6. Search Vehicle ***(const)***

- Search by license plate

7. Zone Revenue ***(const)***

- Sum all parking fees

8. Most Expensive Vehicle ***(const)***

- Find max parking fee

9. Overtime Surcharge

- Apply percentage increase

10. Remove Empty Lot

- Deallocate memory
- Resize array

11. Merge Lots

- Move vehicles
- Delete second lot

12. Total Vehicles

- Display static count

13. Exit Program

- Deallocate ALL memory

SAMPLE OUTPUT FOR MENU

```
===== VEHICLE MANAGEMENT SYSTEM =====
1. Add New Garage
2. Add Vehicle
3. Remove Vehicle by ID
4. Display Garage
5. Find Vehicle with Lowest Fuel
6. Merge Two Garages
7. Remove Empty Garage Automatically
8. Show Total Vehicles
9. Exit

 ==========================================
```

THE ONLINE SHOPPING CART

An e-commerce system is facing serious issues:

- Items remain in the cart even after checkout
- Memory is not freed properly, causing leaks

- Prices sometimes behave incorrectly after discounts

You are tasked with building a **safe and memory-efficient shopping cart system** that eliminates these issues.

TASK

Design a **menu-driven C++ program** using **CLASSES**.

Create a class Product with the following attributes:

```
class Product {
   int   id;
   char  name[40];
   float price;
   static float totalCartValue;  // shared across all products
};
```

SYSTEM REQUIREMENTS

- The cart must start with a **dynamic array of size 3**
- All operations must be implemented using **separate functions**
- Use:
    - **Constructors** (default + parameterized)
    - **static variable** to track total cart value
    - **const function** for display
- Ensure **proper memory handling**:
    - No dangling pointers
    - No memory leaks

FUNCTIONAL REQUIREMENTS

1. Add Product

- Resize array (+1)
- Add product using parameterized constructor
- Update totalCartValue

2. Remove Product

- Remove product by ID
- Shift remaining elements left
- Resize array (-1)
- Update totalCartValue
- Deallocate unused memory

3. Apply Discount

- Select product by ID
- Apply discount percentage
- Update price accordingly
- Update totalCartValue

4. Display Cart ***(const function)***

- Display all product details
- Display total cart value

INVALID PRODUCT RULE

If after applying a discount:

- Product price becomes **negative**

Then:

- The product is considered **invalid**
- It must be **automatically removed from memory**
- Display warning:
  "Invalid Product Removed from Cart!"